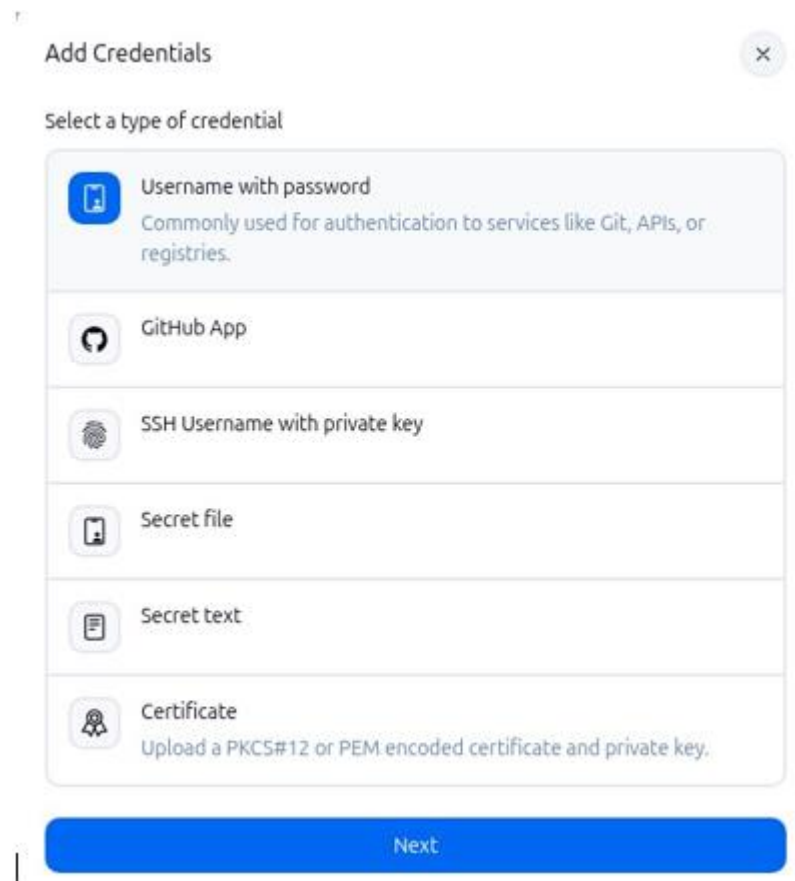


Continuous Integration (CI) Pipeline with Jenkins & Maven/Gradle

Step 1: Login to Jenkins and go to the Credentials page and click on add credentials

Step 2: Add new credentials in Jenkins for CI pipeline. Enter details like username, password, set the scope to Global, and provide a description.



Step 3: now open the terminal and type the command `cd MyMavenApp` to enter the folder and create a **Jenkinsfile**. Paste the code in the Jenkinsfile pipeline {

```
agent any // Use any available agent
tools {
    maven 'Maven' // Ensure this matches the name configured in Jenkins
}
stages {
    stage('Checkout') {
        steps {
            git branch: 'master', url: 'https://github.com/Samiksha201104/MyMavenApp.git'
        }
    }
}
```

```
}  
stage('Build') {  
    steps {  
        sh 'mvn clean package' // Run Maven build  
    }  
}  
stage('Test') {  
    steps {  
        sh 'mvn test' // Run unit tests  
    }  
}  
stage('Run Application') {  
    steps {  
        // Start the JAR application  
        sh 'java -jar target/MyMavenApp-1.0-SNAPSHOT.jar'  
    }  
}  
}  
post {  
    success {  
        echo 'Build and deployment successful!'  
    }  
    failure {  
        echo 'Build failed!'  
    }  
}}
```

Then commit and push it to GitHub

```
samiksha@samiksha:~/Devops6D/MyMavenApp$ git init
Reinitialized existing Git repository in /home/samiksha/Devops6D/MyMavenApp/.git/
samiksha@samiksha:~/Devops6D/MyMavenApp$ git add .
samiksha@samiksha:~/Devops6D/MyMavenApp$ git commit -m "Adding Jenkinsfile"
[master 8735bc6] Adding Jenkinsfile
1 file changed, 47 insertions(+)
create mode 100644 Jenkinsfile
samiksha@samiksha:~/Devops6D/MyMavenApp$ git push -u origin master
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 729 bytes | 729.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0
To https://github.com/Samiksha201104/Devops6D.git
c5925f2..8735bc6 master -> master
Branch 'master' set up to track remote branch 'master' from 'origin'.
samiksha@samiksha:~/Devops6D/MyMavenApp$ █
```

Step 4: Go to Jenkins Dashboard and click on new item , enter the name as MyMavenApp and choose pipeline

 Jenkins / All ▾ / New Item

New Item

Enter an item name

Select an item type

 **Pipeline**
Build, test, and deploy using pipelines. Supports stages, parallel work, and running on multiple agents.

 **Freestyle project**
Classic, general-purpose job type that checks out from up to one SCM, executes build steps serially, followed by post-build steps like archiving artifacts and sending email notifications.

 **Maven project**
Build a maven project. Jenkins takes advantage of your POM files and drastically reduces the configuration.

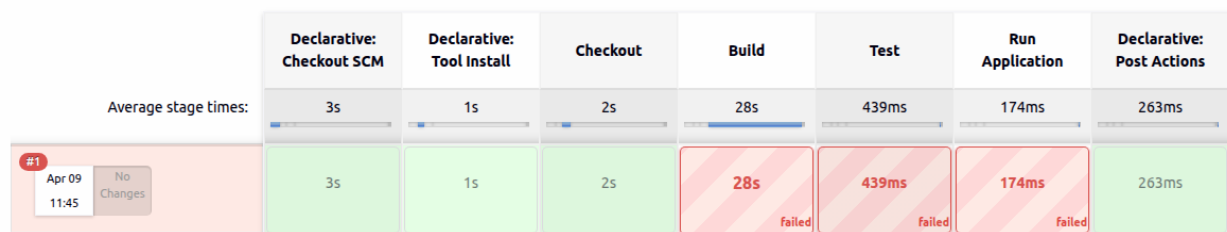
 **Multi-configuration project**

Step 5: in the Jenkins dashboard select MyMavenApp and select build now and then select full stage view

The screenshot shows the Jenkins dashboard. On the left, there's a sidebar with links: 'New Item', 'Build History', 'Project Relationship', and 'Check File Fingerprint'. Below these are two panels: 'Build Queue' (showing 'No builds in the queue.') and 'Build Executor Status' (showing '0/2' executors). The main area displays a table of builds for 'MyMavenApp'. The table has columns: 'S' (Status), 'W' (Workspace), 'Name', 'Last Success', 'Last Failure', and 'Last Duration'. A single build is listed with a green checkmark in the 'S' column, a cloud icon in the 'W' column, and a duration of '33 sec'. Above the table is a filter button labeled 'All'.

Step 6 : Note the stages that failed and check the error by clicking the stage and selecting logs.

MyMavenApp - Stage View



To correct the error open pom.xml and make changes in the code

The original code has the following lines

```
<source>1.7</source>
```

```
<target>1.7</target>
```

Change this code to

```
<source>11</source>
```

```
<target>11</target>
```

Push this back to GitHub and build it again on Jenkins

Gradle with Jenkins

Step 1: in the terminal type the code MyGradleApp to enter the folder .

Type the command gedit JenkinsFile to open the Jenkinsfile and paste the code

```
pipeline {
```

```
    agent any // Use any available agent
```

```
    tools {
```

```
        gradle 'Gradle' // Ensure this matches the name configured in Jenkins
```

```
        jdk 'JDK'
```

```
    }
```

```
    stages {
```

```
        stage('Checkout') {
```

```
            steps {
```

```
                git branch: 'master', url: 'https://github.com/Samiksha201104 /MyGradleApp.git'
```

```
            }
```

```
        }
```

```
        stage('Build') {
```

```
            steps {
```

```
                sh 'gradle build' // Run Gradle build
```

```
            }
```

```
        }
```

```
        stage('Test') {
```

```
            steps {
```

```
                sh 'gradle test' // Run unit tests
```

```
            }
```

```
        }
```

```
        stage('Run Application') {
```

```
            steps {
```

```
                // Start the JAR application
```

```
                sh 'gradle run'
```

```

    }
  }
}

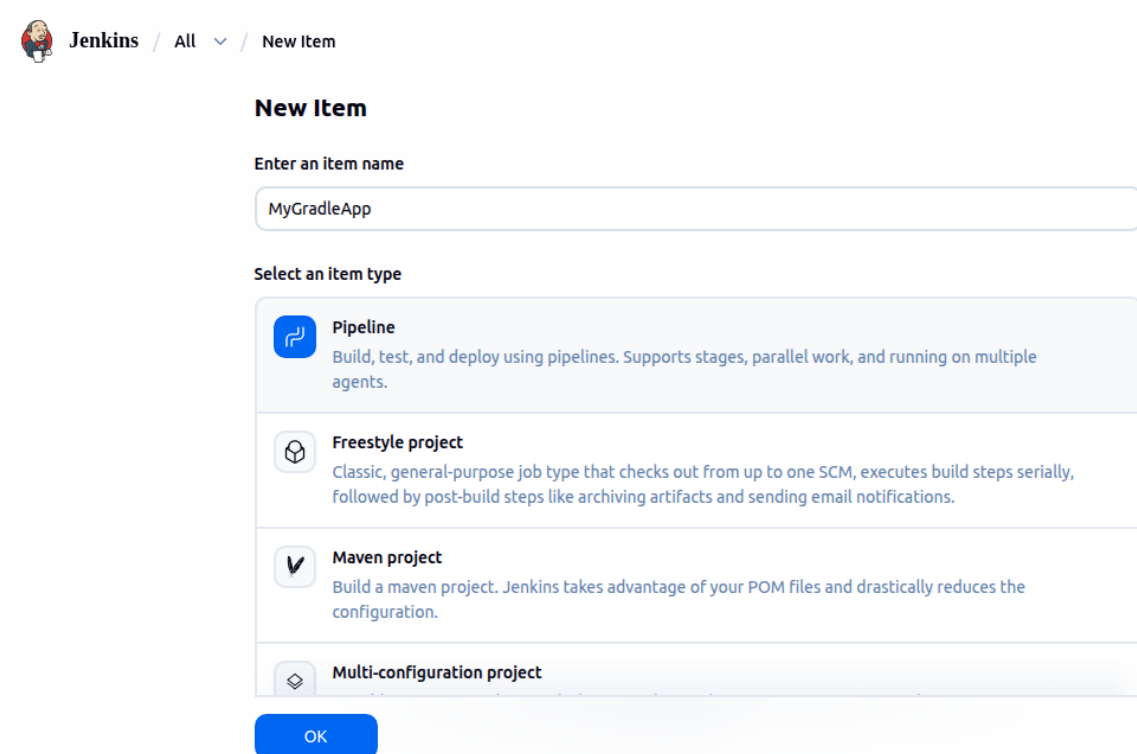
post {
    success {
        echo 'Build and deployment successful!'
    }
    failure {
        echo 'Build failed!'
    }
}
}
}

```

Commit and push to GitHub

Step 2: in Jenkins Dashboard click on New Item

Give the name MyGradleApp and select Pipeline



Jenkins / All / New Item

New Item

Enter an item name

MyGradleApp

Select an item type

- Pipeline**
Build, test, and deploy using pipelines. Supports stages, parallel work, and running on multiple agents.
- Freestyle project**
Classic, general-purpose job type that checks out from up to one SCM, executes build steps serially, followed by post-build steps like archiving artifacts and sending email notifications.
- Maven project**
Build a maven project. Jenkins takes advantage of your POM files and drastically reduces the configuration.
- Multi-configuration project**

OK

In configure scroll down to Pipeline and choose pipeline script from SCM, give the repository url and choose GitHub Credentials

Jenkins / MyGradleApp / Configure

Configure

- General
- Triggers
- Pipeline
- Advanced

Pipeline

Define your Pipeline using Groovy directly or pull it from source control.

Definition

Pipeline script from SCM

SCM ?

Git

Script Path ?

Jenkinsfile

☒ Lightweight checkout ?

[Pipeline Syntax](#)

Advanced

Save Apply

Jenkins / MyGradleApp / Configure

Configure

- General
- Triggers
- Pipeline
- Advanced

Repositories ?

Repository URL ?

https://github.com/Samiksha201104/MyGradleApp.git

Credentials ?

GitHub Credentials

+ Add

Advanced

+ Add Repository

Branches to build ?

Then click on apply and save

Jenkins / MyGradleApp / Configure

Configure

- General
- Triggers
- Pipeline
- Advanced

repository browser ?

(Auto)

Additional Behaviours

+ Add

Script Path ?

Jenkinsfile

☒ Lightweight checkout ?

[Pipeline Syntax](#)

Advanced

Advanced

Save Apply

Step 3: go to Jenkins Dashboard select the MyGradleApp pipeline and select Build Now. After build select Full Stage View

Jenkins

+ New Item

Build History

Project Relationship

Check File Fingerprint

Build Queue

No builds in the queue.

Build Executor Status

0/2

All

+

S	W	Name ↓	Last Success	Last Failure	Last Duration
		MyGradleApp	N/A	N/A	N/A
		MyMavenApp		23 hr #1	33 sec

Icon: S M L

Changes

Build Now

Configure

Delete Pipeline

Full Stage View

Stages

Rename

Pipeline Syntax

Add description

Once the errors are corrected Build again

Jenkins

MyGradleApp

Full Stage View

Search

Settings

User

MyGradleApp - Stage View

Average stage times:

Declarative: Checkout SCM	Declarative: Tool Install	Checkout	Build	Test	Run Application	Declarative: Post Actions
3s	536ms	2s	2min 50s	51s	15s	7s

	Declarative: Checkout SCM	Declarative: Tool Install	Checkout	Build	Test	Run Application	Declarative: Post Actions
#4 Apr 10 21:49 1 commit	3s	314ms	2s	1min 40s	1min 14s	50s	13s
#3 Apr 10 21:29 2 commits	2s	301ms	2s	4min 48s	1min 36s failed	8s failed	8s
#2 Apr 10 21:10 2 commits	4s	743ms	1s	4min 43s failed	32s failed	2s failed	6s
#1 Apr 10 11:28 No Changes	2s	786ms	2s	10s failed	654ms failed	204ms failed	522ms

MyMavenPracticeAutomtion

Step 1: now open the terminal and type the command **cd MyMavenPracticeAutomationApp** to enter the folder and create a **Jenkinsfile** .Paste the code in the Jenkinsfile

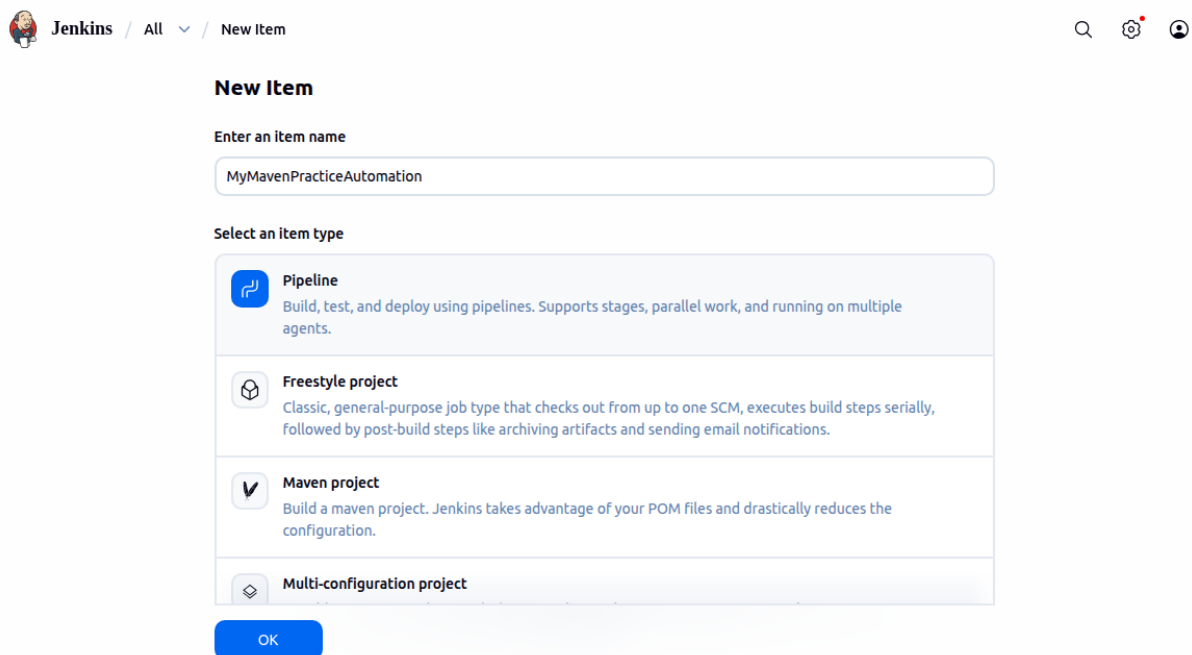
```
pipeline {
    agent any // Use any available agent
    tools {
        maven 'Maven' // Ensure this matches the name configured in Jenkins
    }
    stages {
        stage('Checkout') {
            steps {
                git branch: 'master', url:
'https://github.com/Samiksha201104/MyMavenPracticeAutomationApp.git'
            }
        }
        stage('Build') {
            steps {
                sh 'mvn clean package' // Run Maven build
            }
        }
        stage('Test') {
            steps {
                sh 'mvn test' // Run unit tests
            }
        }
        stage('Run Application') {
            steps {
                // Start the JAR application
                sh 'java -jar target/MyMavenApp-1.0-SNAPSHOT.jar'
            }
        }
    }
    post {
        success {
```

```

    echo 'Build and deployment successful!'
}
failure {
    echo 'Build failed!'
}}}
```

Then commit and push it to GitHub

Step 2:in the Jenkins Dashboard click on new item type the name of the item and select Pipeline



Jenkins / All / New Item

New Item

Enter an item name

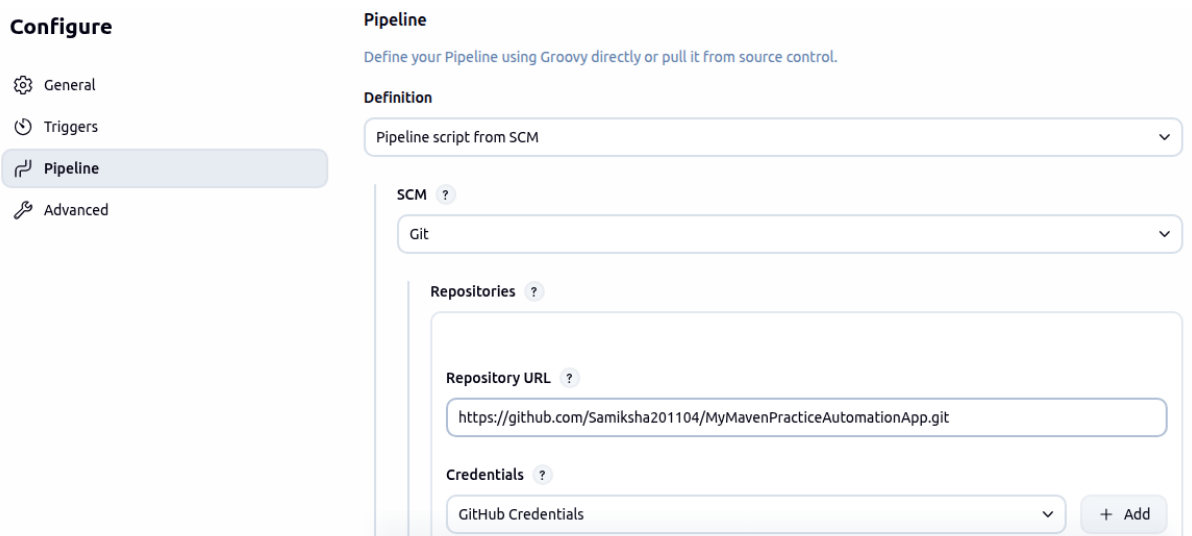
MyMavenPracticeAutomation

Select an item type

- Pipeline**
Build, test, and deploy using pipelines. Supports stages, parallel work, and running on multiple agents.
- Freestyle project
Classic, general-purpose job type that checks out from up to one SCM, executes build steps serially, followed by post-build steps like archiving artifacts and sending email notifications.
- Maven project
Build a maven project. Jenkins takes advantage of your POM files and drastically reduces the configuration.
- Multi-configuration project

OK

In configure scroll down to Pipeline and choose pipeline script from SCM,give the repository url and choose GitHub Credentials



Configure

- General
- Triggers
- Pipeline**
- Advanced

Pipeline

Define your Pipeline using Groovy directly or pull it from source control.

Definition

Pipeline script from SCM

SCM ?

Git

Repositories ?

Repository URL ?

https://github.com/Samiksha201104/MyMavenPracticeAutomationApp.git

Credentials ?

GitHub Credentials + Add

Step 3 : in the Dashboard click on the MyMavenPracticeAutomationApp item and select Build Now and then Full Stage view

All

</> Changes

▶ Build Now

⚙️ Configure

🗑️ Delete Pipeline

🔍 Full Stage View

📁 Stages

✎ Rename

❓ Pipeline Syntax

S

W

Name ↓

✔️☁️

MyGradleApp

✔️☁️

MyMavenApp

⋮☀️

MyMavenPracticeAutomationApp

▼

Duration

Last Duration

1 min

5 min 51 sec

33 sec

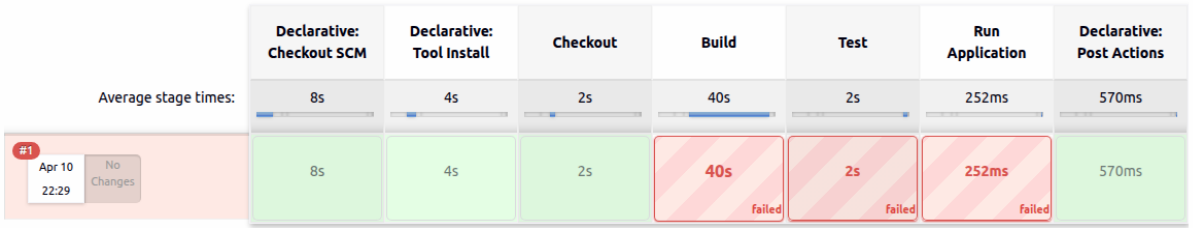
N/A

▶

▶

▶

MyMavenPracticeAutomationApp - Stage View



To correct the errors change the code in the pom.xml

<source>1.7</source>

To

<source>17</source>

<target>1.7</target>

<target>17</target>

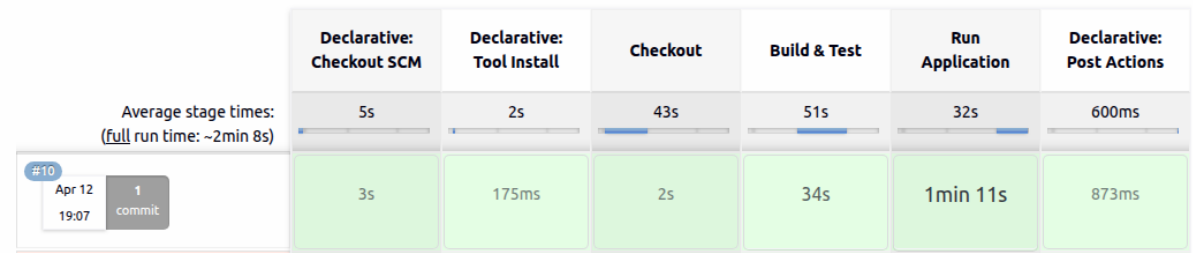
Save and Build again in Jenkins

 Jenkins

MyMavenPracticeAutoma...

Full Stage View

MyMavenPracticeAutomationApp - Stage View



MyMavenExerciseAutomationApp

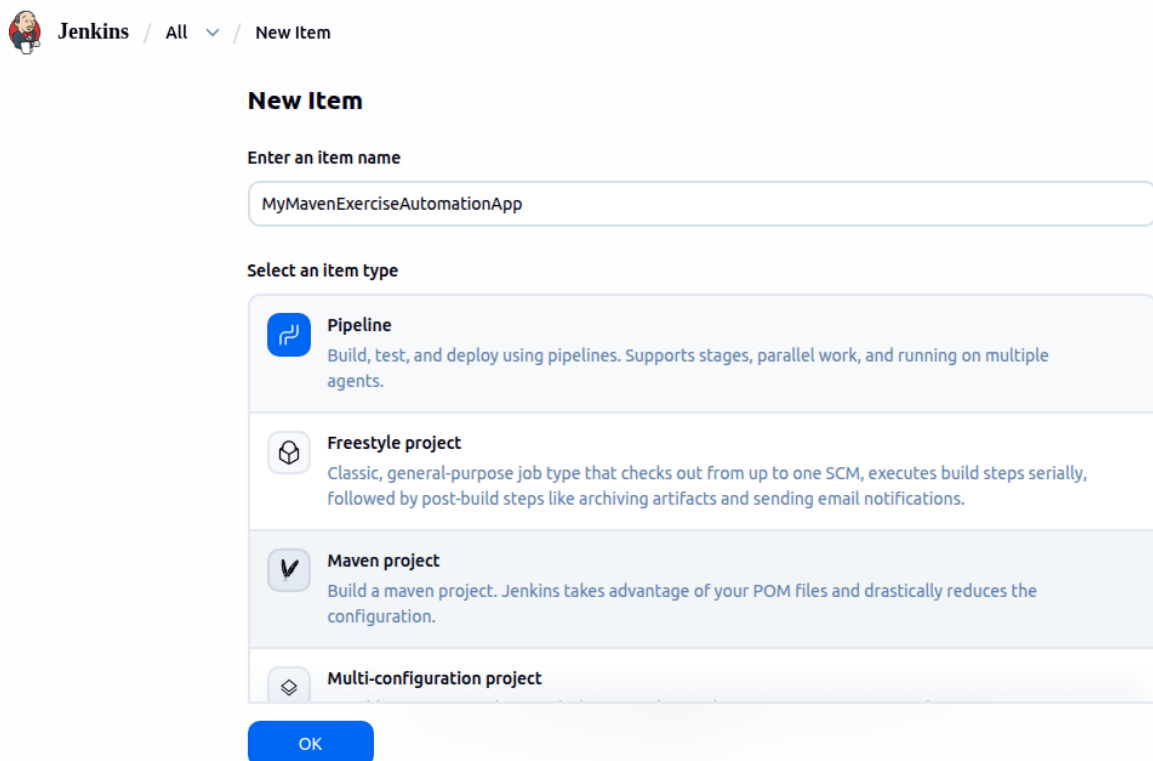
Step 1: now open the terminal and type the command **cd MyMavenExerciseAutomationApp** to enter the folder and create a **Jenkinsfile** .Paste the code in the Jenkinsfile

```
pipeline {
    agent any // Use any available agent
    tools {
        maven 'Maven' // Ensure this matches the name configured in Jenkins
    }
    stages {
        stage('Checkout') {
            steps {
                git branch: 'master', url:
'https://github.com/Samiksha201104/MyMavenExerciseAutomationApp.git'
            }
        }
        stage('Build') {
            steps {
                sh 'mvn clean package' // Run Maven build
            }
        }
        stage('Test') {
            steps {
                sh 'mvn test' // Run unit tests
            }
        }
        stage('Run Application') {
            steps {
                // Start the JAR application
                sh 'java -jar target/MyMavenApp-1.0-SNAPSHOT.jar'
            }
        }
    }
    post {
        success {
```

```
    echo 'Build and deployment successful!'
}
failure {
    echo 'Build failed!'
}}
```

Then commit and push it to GitHub

Step 2:in the Jenkins Dashboard click on new item type the name of the item and select Pipeline



Jenkins / All / New Item

New Item

Enter an item name

MyMavenExerciseAutomationApp

Select an item type

-  **Pipeline**
Build, test, and deploy using pipelines. Supports stages, parallel work, and running on multiple agents.
-  **Freestyle project**
Classic, general-purpose job type that checks out from up to one SCM, executes build steps serially, followed by post-build steps like archiving artifacts and sending email notifications.
-  **Maven project**
Build a maven project. Jenkins takes advantage of your POM files and drastically reduces the configuration.
-  **Multi-configuration project**

OK

In configure scroll down to Pipeline and choose pipeline script from SCM,give the repository url and choose GitHub Credentials

Jenkins / MyMavenExerciseAutoma... / Configure

Configure

- General
- Triggers
- Pipeline**
- Advanced

Definition

Pipeline script from SCM

SCM

Git

Repositories

Repository URL: `https://github.com/Samiksha201104/MyMavenExerciseAutomationApp.git`

Credentials: **GitHub Credentials** + Add

Advanced

Save Apply

Step 3 : in the Dashboard click on the MyMavenExerciseAutomationApp item and select Build Now and then Full Stage view

Jenkins / MyMavenExerciseAutoma... / Full Stage View

MyMavenExerciseAutomationApp - Stage View

Average stage times: (full run time: ~10min 39s)

	Declarative: Checkout SCM	Declarative: Tool Install	Checkout	Build	Run Application	Declarative: Post Actions
#5 Apr 12 19:55 2 commits	1s	229ms	1s	3min 2s	40s	1s
#4 Apr 12 19:48 1 commit	2s	327ms	2s	7min 59s	1min 54s	3s
#3 Apr 12 19:44 3 commits	1s	241ms	2s	59s	4s failed	144ms
	1s	120ms	1s	7s failed	1s failed	177ms

MyMavenGuavaApp

Step 1: now open the terminal and type the command **cd MyMavenGuavaApp** to enter the folder and create a **Jenkinsfile** .Paste the code in the Jenkinsfile

```
pipeline {
    agent any // Use any available agent
    tools {
        maven 'Maven' // Ensure this matches the name configured in Jenkins
    }
    stages {
        stage('Checkout') {
            steps {
                git branch: 'master', url:
'https://github.com/Samiksha201104/MyMavenGuavaApp.git'
            }
        }
        stage('Build') {
            steps {
                sh 'mvn clean package' // Run Maven build
            }
        }
        stage('Test') {
            steps {
                sh 'mvn test' // Run unit tests
            }
        }
        stage('Run Application') {
            steps {
                // Start the JAR application
                sh 'java -jar target/MyMavenGuavaApp-1.0-SNAPSHOT.jar'
            }
        }
    }
    post {
        success {
```

```

    echo 'Build and deployment successful!'
}
failure {
    echo 'Build failed!'
}}}
```

Then commit and push it to GitHub

Step 2: in the Jenkins Dashboard click on new item type the name of the item and select Pipeline

Jenkins / All / New Item

New Item

Enter an item name

MyMavenGuavaApp

Select an item type

- Pipeline**
Build, test, and deploy using pipelines. Supports stages, parallel work, and running on multiple agents.
- Freestyle project**
Classic, general-purpose job type that checks out from up to one SCM, executes build steps serially, followed by post-build steps like archiving artifacts and sending email notifications.
- Maven project**
Build a maven project. Jenkins takes advantage of your POM files and drastically reduces the configuration.
- Multi-configuration project**

OK

In configure scroll down to Pipeline and choose pipeline script from SCM, give the repository url and choose GitHub Credentials

Jenkins / MyMavenGuavaApp / Configure

Configure

Pipeline script from SCM

SCM

Git

Repositories

Repository URL

https://github.com/Samiksha201104/MyMavenGuavaApp.git

Please enter Git repository.

Credentials

GitHub Credentials

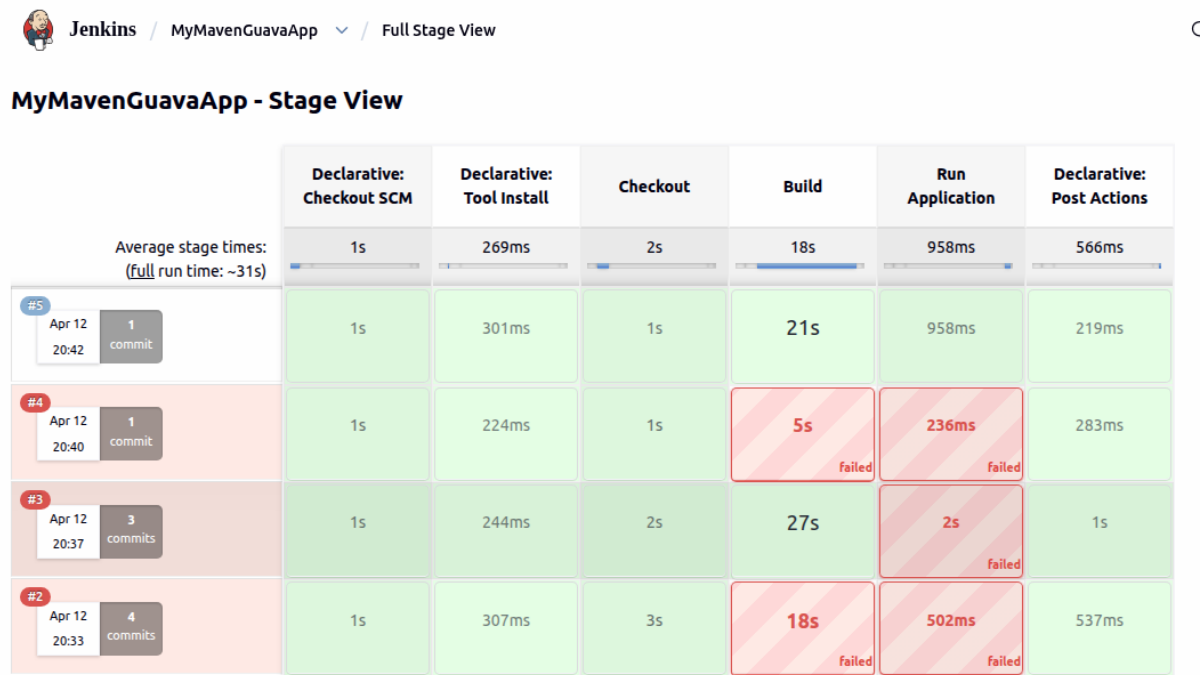
+ Add

Advanced

Save Apply

Step 3 : in the Dashboard click on the MyMavenGuavaApp item and select Build Now and then Full Stage view

Correct the errors and build again on jenkins till all the stages are successful



MyMavenToGradle

Step 1: now open the terminal and type the command **cd MyMavenToGradleApp** to enter the folder and create a **Jenkinsfile** .Paste the code in the Jenkinsfile

pipeline {

agent any // Use any available agent

tools {

maven 'Maven' // Ensure this matches the name configured in Jenkins

}

stages {

stage('Checkout') {

steps {

git

branch:

'master',

url:

'https://github.com/Samiksha201104/MyMavenGradleApp.git'

}

}

stage('Build') {

steps {

```

        sh 'mvn clean package' // Run Maven build
    }
}
stage('Test') {
    steps {
        sh 'mvn test' // Run unit tests
    }
}
stage('Run Application') {
    steps {
        // Start the JAR application
        sh 'java -jar target/MyMavenGradleApp-1.0-SNAPSHOT.jar'
    }
}
}
post {
    success {
        echo 'Build and deployment successful!'
    }
    failure {
        echo 'Build failed!'
    }
}
}
}

```

Then commit and push it to GitHub

Step 2: in the Jenkins Dashboard click on new item type the name of the item and select Pipeline

New Item

Enter an item name

MyMavenToGradle

Select an item type



Pipeline

Build, test, and deploy using pipelines. Supports stages, parallel work, and running on multiple agents.



Freestyle project

Classic, general-purpose job type that checks out from up to one SCM, executes build steps serially, followed by post-build steps like archiving artifacts and sending email notifications.



Maven project

Build a maven project. Jenkins takes advantage of your POM files and drastically reduces the configuration.



Multi-configuration project

OK

In configure scroll down to Pipeline and choose pipeline script from SCM, give the repository url and choose GitHub Credentials

Jenkins / MyMavenToGradle / Configure

Q ? ?

Configure

General

Triggers

Pipeline

Advanced

Definition

Pipeline script from SCM

SCM ?

Git

Repositories ?

Repository URL ?

https://github.com/Samiksha201104/MyMavenToGradle.git

Credentials ?

GitHub Credentials

+ Add

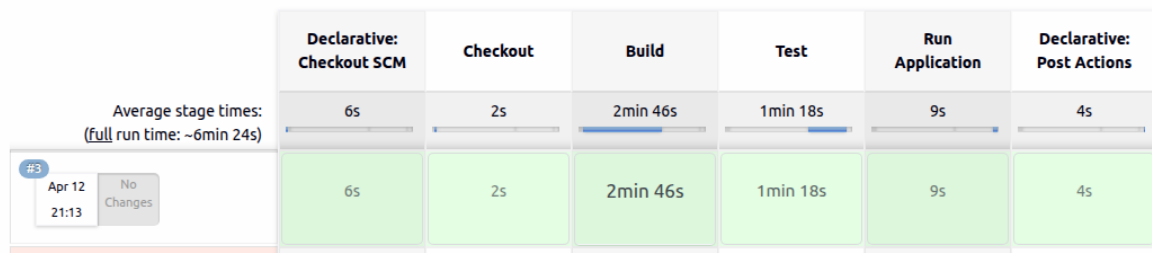
Advanced

Save Apply

Step 3 : in the Dashboard click on the MyMavenToGradleApp item and select Build Now and then Full Stage view

Correct the errors and build again on jenkins till all the stages are successful

MyMavenToGradle - Stage View



MyMavenSeleniumApp

Step 1: now open the terminal and type the command **cd MyMavenSeleniumApp** to enter the folder and create a **Jenkinsfile** .Paste the code in the Jenkinsfile

pipeline {

agent any // Use any available agent

tools {

maven 'Maven' // Ensure this matches the name configured in Jenkins

}

stages {

stage('Checkout') {

steps {

git branch: 'master', url:

'https://github.com/Samiksha201104/MyMavenSeleniumApp.git'

}

}

stage('Build') {

steps {

sh 'mvn clean package' // Run Maven build

}

}

stage('Test') {

steps {

sh 'mvn test' // Run unit tests

}

}

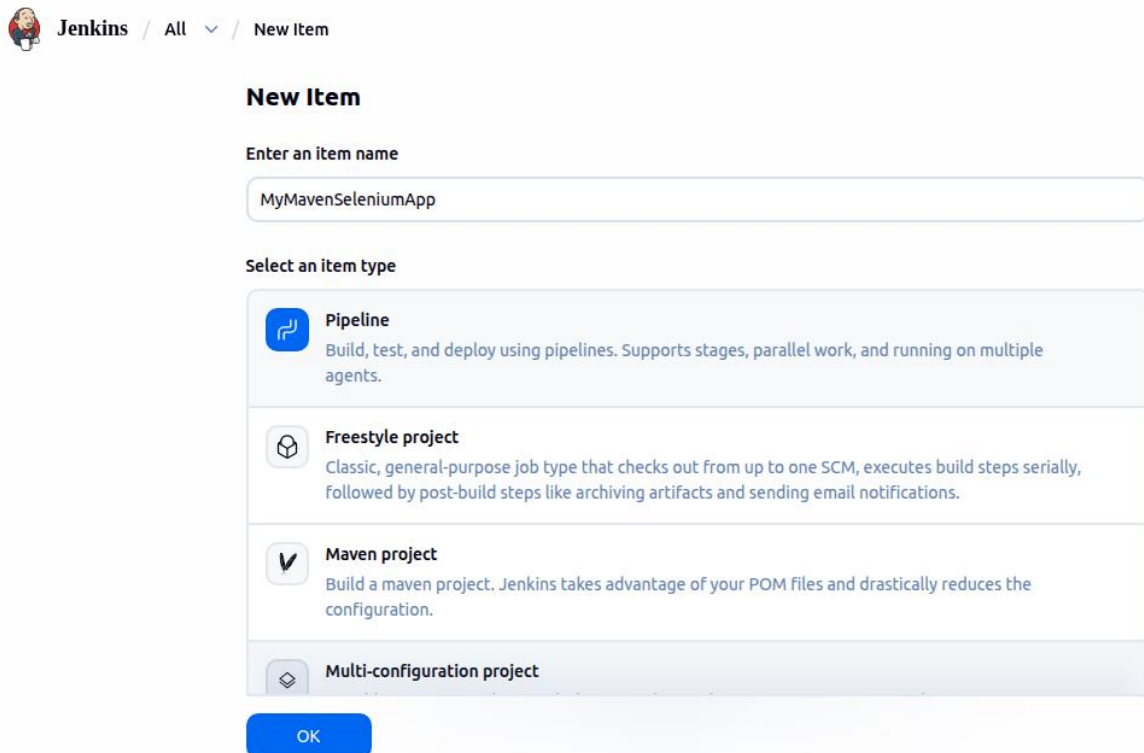
```

stage('Run Application') {
    steps {
        // Start the JAR application
        sh 'java -jar target/MyMavenSeleniumApp-1.0-SNAPSHOT.jar'
    }
}
}
post {
    success {
        echo 'Build and deployment successful!'
    }
    failure {
        echo 'Build failed!'
    }
}
}

```

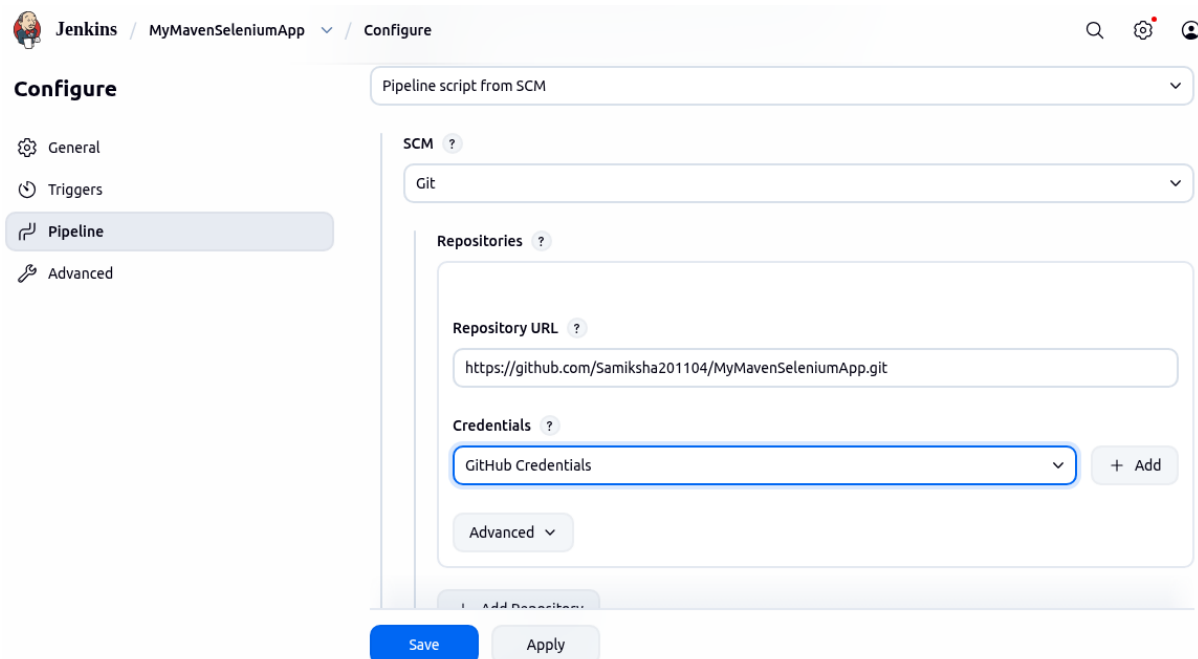
Then commit and push it to GitHub

Step 2: in the Jenkins Dashboard click on new item type the name of the item and select Pipeline



The screenshot shows the Jenkins 'New Item' page. At the top, there's a navigation bar with the Jenkins logo, 'Jenkins', and a dropdown menu showing 'All' and 'New Item'. Below this, the title 'New Item' is displayed. A text input field labeled 'Enter an item name' contains the text 'MyMavenSeleniumApp'. Underneath, a section titled 'Select an item type' lists four options: 'Pipeline' (with a blue icon and description), 'Freestyle project' (with a cube icon and description), 'Maven project' (with a checkmark icon and description), and 'Multi-configuration project' (with a diamond icon). The 'Maven project' option is selected, indicated by a checkmark in its icon. At the bottom of the list is an 'OK' button.

In configure scroll down to Pipeline and choose pipeline script from SCM, give the repository url and choose GitHub Credentials



The image shows the Jenkins configuration page for a project named 'MyMavenSeleniumApp'. The 'Configure' tab is selected, and the 'Pipeline script from SCM' option is chosen. The 'SCM' dropdown is set to 'Git'. The 'Repository URL' is 'https://github.com/Samiksha201104/MyMavenSeleniumApp.git'. The 'Credentials' dropdown is set to 'GitHub Credentials'. The 'Advanced' section is expanded, showing 'Add Repository' and 'Save' buttons.

Step 3 : in the Dashboard click on the MyMavenToGradleApp item and select Build Now and then Full Stage view

Correct the errors and build again on jenkins till all the stages are successful

